

Algorithms, AI & Data Science II

Franziska Heeg

Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

Motivation

- ▶ we introduced **propositional logic**, a context-free formal language that can be used to **represent knowledge**
- ▶ we studied algorithms to decide the **satisfiability** of propositional formula and to **infer conclusions**
- ▶ foundation for **automatic reasoning** based on a knowledgebase and **symbolic AI**
- ▶ is propositional logic **expressive enough** to represent knowledge about any given domain?
- ▶ can we use propositional logic to **infer any conclusion** that we can intuitively infer?

X_0	X_1	$X_0 \implies X_1$
False	False	True
False	True	True
True	False	False
True	True	True

Notes:

- **Lecture L08: First-Order Predicate Logic** 26.05.2025
- **Educational objective:** We motivate limitations of propositional logic for knowledge representation. We address them by extending propositional logic by quantors and predicates.
 - First-Order Predicate Logic
 - Formula Evaluation and Semantics
 - Satisfiability in First-Order Logic
 - Inference in First-Order Logic
- **Exercise sheet 04:** Logical Inference due 02.06.2025
- **Handout version:** 2025/05/26 09:32:25

Notes:

Propositional logic and knowledge representation

- ▶ let us use **propositional logic** to represent knowledge in a simple example
- ▶ we use **propositional variables** to capture atomic propositions
- ▶ we use **logical connectives** to represent non-atomic propositions
- ▶ can you infer the truth value of HothoTeachesLecture?
- ▶ propositional logic lacks means to express statements about ...
 1. **sets of entities** (e.g. professors, lectures)
 2. **predicates on entities** (e.g. Professor(Hotho))
 3. **relations between entities** (e.g. Teaches(Glavas, AKIDS1))

example knowledgebase

- ▶ Ingo Scholtes is a professor
- ▶ Andreas Hotho is a professor
- ▶ Franziska Heeg is a PhD student
- ▶ Goran Glavas is a professor
- ▶ Goran Glavas teaches AKIDS1
- ▶ Ingo Scholtes teaches AKIDS2
- ▶ Franziska Heeg teaches AKIDS2
- ▶ **Every professor teaches at least one lecture**
- ▶ **Every lecture is taught by at least one professor**
- ▶ **Not everyone teaching a lecture is a professor**

representation in propositional logic

1. ScholtesIsProfessor
2. HothoIsProfessor
3. HeegIsPhDStudent
4. GlavasIsProfessor
5. GlavasTeachesAKIDS1
6. HeegTeachesAKIDS2
7. ScholtesTeachesAKIDS2
8. isProfessor $\implies$ teachesLecture
9. isLecture $\implies$ taughtByProfessor
10. teachesLecture $\nRightarrow$ isProfessor

From propositions to predicates

propositional logic

We consider two independent **atomic propositions**:

- ▶ “Goran Glavas is a professor”
- ▶ “Ingo Scholtes is a professor”

and represent them by propositional variables X_0 and X_1 .

We use logical connectives to form compound propositions like

$$\begin{aligned} X_0 \wedge X_1 \\ X_0 \iff X_1 \\ \dots \end{aligned}$$

first-order predicate logic

We define **unary logical predicate** $P(x)$ indicating that entity x is a professor. We consider **instantiations** of predicate P

- ▶ $P(x_0)$
- ▶ $P(x_1)$

where x_0 and x_1 represent **entities** “Goran Glavas” and “Ingo Scholtes”, respectively. We use logical connectives to form compound propositions like

$$\begin{aligned} P(x_0) \wedge P(x_1) \\ P(x_0) \iff P(x_1) \\ \dots \end{aligned}$$

- ▶ different from propositional logic, **variables represent entities** rather than atomic propositions
- ▶ like in propositional logic, we use **logical connectives** to form compound propositions
- ▶ instantiation of **logical predicates** yields propositions that are either True or False
- ▶ how can we capture that a professor teaches a certain lecture?

Notes:

Notes:

- First-order logic = predicate logic - quantificational logic = first-order predicate calculus

Relations and quantifiers

- ▶ we want to make statements about **relations between entities**, e.g. “Goran Glavas teaches AKIDS1”
- ▶ we generalize unary predicates to **k -nary logical predicates** $R(x_0, \dots, x_k)$, which indicates that $(x_0, \dots, x_k) \in R$ for relation R
- ▶ so far, we only made statements about **specific entities**, i.e. about “Goran Glavas”
- ▶ how can we express “**patterns**” like, e.g., “every professor teaches at least one lecture”?
- ▶ we can express that a predicate holds **for all** entities or that an entity **exists** for which predicate holds
- ▶ “for all” and “exists” correspond to **universal ($\forall$) and existential ($\exists$) quantifier**, respectively

k -nary predicates

Define binary predicate $T(x, y)$ indicating that professor x teaches lecture y . We consider instantiations of predicate T , e.g.

1. $T(x_0, y_0)$
2. $T(x_1, y_1)$

where x_0 and x_1 represent entities “Goran Glavas” and “Ingo Scholtes” and y_0 and y_1 represent entities AKIDS1 and AKIDS2.

quantifier

We define unary predicate $L(x)$ indicating that x is a lecture. Using unary predicate $P(x)$ indicating that x is a professor and binary predicate $T(x, y)$ indicating that x teaches lecture y we use the **universal and existential quantifier** to form compound propositions:

$$\forall x P(x) \implies \exists y L(y) \wedge T(x, y)$$

$$\forall y L(y) \implies \exists x P(x) \wedge T(x, y)$$

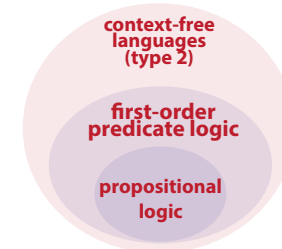
First-order predicate logic

- ▶ first-order predicate logic (FOPL) extends operators in propositional logic by **quantifiers** $\forall, \exists$ and adds **predicates** over variables $\rightarrow$ G Frege, 1879
 1. “first-order” logic $\approx$ **quantifiers over entities**
 2. “second-order” logic $\approx$ **quantifiers over relations**
 3. “higher-order” logic $\approx$ **quantifiers over arbitrary subsets**
 4. “zero-order” logic $\approx$ **propositional logic**
- ▶ first-order logic is **context-free formal language** to represent knowledge about elements of sets
- ▶ first-order logic includes propositional logic and is **truly more expressive than propositional logic**
- ▶ first-order logic in domain of sets $(\mathbb{N}, \mathbb{Z}, \mathbb{R}^d, \dots)$ is “**language of thought**” in mathematics (and basis for all “hard sciences”)



Gottlob Frege, 1848 – 1925

image credit: Wikimedia Commons, public domain



Notes:

Notes:

Syntax of first-order predicate logic

definitions

- ▶ A **variable** represents arbitrary entities of an underlying set (which we call universe). We denote variables by lower-case letters, e.g. $x_0, x_1, \dots$
- ▶ A **constant** represents a specific entity in the universe, e.g. number 5, string "dog", person "Andreas Hotho", as well as **logical constants** `True`, `False`
- ▶ **k-nary logical predicates** are denoted by identifiers starting with capital letter, e.g. $P(x_0, \dots, x_{k-1})$. **Built-in predicates** $=, <, >, \geq, \neq$ denote the corresponding binary relations and we write $5 < x_0$ instead of $<(5, x_0)$.
- ▶ **k-nary functions** are denoted by identifiers starting with non-capital letter, e.g. $f(x_0, \dots, x_{k-1})$. **Built-in functions** $+, -, *, /, \dots$ denote the corresponding arithmetic operations. We write $x_0 + 1$ instead of $+(x_0, 1)$.
- ▶ We call constants and variables **terms**. If t_i are terms and f is a function, then $f(t_0, \dots, t_{k-1})$ is a term. **Atomic formulas** have the form $P(t_0, \dots, t_{k-1})$ where P is a predicate and t_i are terms.
- ▶ $\wedge, \vee, \neg, \implies$, and $\iff$ denote the corresponding **logical connectives** in propositional logic.
- ▶ If α and β are formulas, then $\neg\alpha, \alpha \wedge \beta, \alpha \vee \beta, \alpha \implies \beta$ and $\alpha \iff \beta$ are formulas.
- ▶ The **universal or existential quantifier** $\forall$ and $\exists$ is immediately followed by a variable, e.g. $\forall x$ or $\exists x$. If α is a formula, then $\exists x\alpha$ and $\forall x\alpha$ are formulas for any variable x .

example

$$\exists x (P(x) \wedge \neg Q(y, x) \implies 5 < f(x))$$

x , and y are variables
 5 is a constant
 P, Q , and $<$ are predicates
 f is a function

$5, x, y$, and $f(x)$ are terms
 $P(x), Q(y, x), 5 < f(x)$ are atomic formulas

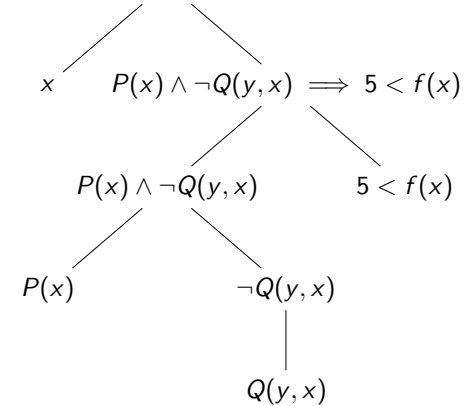
$\wedge, \neg, \implies$ are logical connectives
 $P(x) \wedge \neg Q(y, x) \implies 5 < f(x)$ is a formula

$\exists$ is the existential quantifier
 $\exists x (P(x) \wedge \neg Q(y, x) \implies 5 < f(x))$ is a formula

Recursive evaluation and operator precedence

- ▶ like for propositional logic, to evaluate formula **we recursively apply semantics** of predicates, quantifiers, and logical connectives
- ▶ formula defines **tree**, where nodes represent (sub-)formulas, links represent logical connectives and quantifiers, and leafs represent atomic formulas
- ▶ order of evaluation is defined by **operator precedence** $\neg, \wedge, \vee, \forall, \exists, \implies, \iff$
- ▶ we can use **brackets to change evaluation order** (or avoid ambiguity)

$$\exists x [P(x) \wedge \neg Q(y, x) \implies 5 < f(x)]$$



example 2

$$\exists x [P(x) \wedge \neg Q(20, x) \implies 5 < f(x)]$$

Notes:

Notes:

- In example 1 above, note that – due to fact that $\exists$ has stronger precedence than $\implies$ – the scope of the existential quantifier only comprises the antecedent and not the consequent of the implication, i.e. the x in $5 < f(x)$ is not bound to the quantifier but a so-called “free” variable (see slide 11) that happens to have the same name as the variable x in the scope of the existential quantifier.

Group Exercise 08-01

1/3

Which of the following are syntactically correct formulas in first-order predicate logic.

1. $14^2 < 47$

correct, $<$ is built-in binary predicate. 14, 2 and 47 are constant terms, and 14^2 is short for $f(14, 2)$ with built-in function $f(x, y) = x^y$

2. $\forall x \exists y x < y$

correct. We can have nested quantifiers, where $\exists y x < y$ is the inner formula in the quantification scope of the universal quantifier

3. $L(M(H), F \vee G)$

wrong, F and G are predicates without terms and H is a predicate, not a variable.

4. $\forall x \text{Smart}(x) \wedge \neg \text{Alive}(x) \implies \text{Famous}(x)$

correct

5. $\exists x \forall P P(x)$

wrong, we cannot use quantifiers on predicates (but valid in higher-order logic).

Notes:

Group Exercise 08-01

2/3

Give formulas in first-order predicate logic that represent the following statements in natural language.

6. Anyone who is dedicated learns predicate logics.

$$\forall x \text{Dedicated}(x) \implies \text{Learns}(x, \text{'predicate logics'})$$

7. No man is independent.

$$\neg \exists x (\text{Man}(x) \wedge \text{Independent}(x))$$

8. Dogs that bark do not bite.

$$\text{Dog}(x) \wedge \text{Bark}(x) \implies \neg \text{Bites}(x)$$

9. All yellow mushrooms are poisonous.

$$\forall x \text{Mushroom}(x) \wedge \text{Yellow}(x) \implies \text{Poisonous}(x)$$

Notes:

Group Exercise 08-01

3/3

Give formulas in first-order predicate logic that represent the following statements in natural language.

10. Not all women can sing.

$\neg \forall x (Woman(x) \implies Sing(x))$
alternatively we could write
 $\exists x Woman(x) \wedge \neg Sing(x)$

11. Every teenager owns at least one smartphone.

$\forall x (Teenager(x) \implies \exists y (SmartPhone(y) \wedge Owns(x, y)))$

12. Ingo likes everyone who does not like himself.

$\forall x (\neg Likes(x, x) \implies Likes('Ingo', x))$

Notes:

Formula evaluation in first-order logic

- ▶ in **propositional logic**, we can evaluate truth value of formula for any **assignment of truth values to propositional variables** → truth table enumeration
- ▶ in first-order predicate logic we must distinguish between **two types of variables**
 - ▶ variables that are quantified are called **bound**
 - ▶ variables that are not quantified are called **free**
 - ▶ same variable can **occur both as bound and free**
- ▶ formula without free variables is called **sentence**
- ▶ we can **evaluate truth value of sentences** for underlying set/universe (and given predicate semantics)
- ▶ for formulas with free variables, we can evaluate truth value of formula for any **assignment of entity to free variable** → assignment of variable y in example 1

example 1

$$\exists x (P(x) \wedge \neg Q(y, x)) \implies 5 < x$$

variable x **is bound** because it is quantified by $\exists$
variable y **is free** because it does not appear with a quantifier, i.e. formula is **not a sentence**

example 2

$$\forall x \exists y \quad x < y$$

variables x and y **are bound** because they are quantified by $\forall$ and $\exists$. Formula is **a sentence** that evaluates to true for the universe $\mathbb{N}$

example 3

$$\exists x P(x) \wedge \neg Q(20, x) \implies 5 < f(x)$$

x is both a bound and a free variable, i.e. formula is **not a sentence**

Notes:

Semantics of first-order predicate logic

- can we evaluate truth value of the following formula in first-order predicate logic?
- semantics of FOPL formula F depends on ...
 1. underlying **universe**,
 2. assignment of **constants** to entities of universe
 3. **semantics of predicates**,
 4. **semantics of functions**, and
 5. **assignment of free variables** to entities of universe (iff F is not a sentence)
- we call this a **structure** $\mathcal{A}$ for F
- structure $\mathcal{A}$ allows to **evaluate truth value** $\mathcal{A}(F)$ of formula F in first-order predicate logic
- structure $\mathcal{A}$ for which F evaluates to True is said to **satisfy** F (we write $\mathcal{A} \models F$ and call $\mathcal{A}$ "model" for F)

example

$$\exists x(P(x) \wedge \neg Q(y, x) \implies 5 < f(x))$$

example structure $\mathcal{A}$

universe $U = \{1, 2, 3, 4, 5\}$

predicate semantics

$P = \{1, 2, 3\}$

and

$Q = \{(1, 4), (2, 5), (4, 5)\}$

function semantics $f(x) := x - 1$

free variable assignment $y \leftarrow 2$

(where constants 1, 2, 3, 4, 5 are implicitly assigned to corresponding entities of universe)

Group Exercise 08-02

1/2

- Evaluate the truth value of the first-order predicate logic formula F

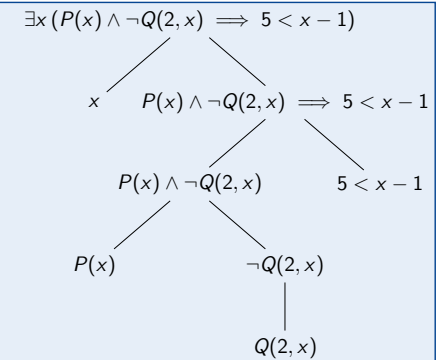
$$\exists x(P(x) \wedge \neg Q(y, x) \implies 5 < f(x))$$

for the structure $\mathcal{A}$ given on the previous slide. Is $\mathcal{A}$ a model for F ?

- we first **substitute free variable** y with entity $2 \in U$ (and obtain a sentence)
- we apply **semantics of function** $f(x) := x - 1$
- we test whether $P(x) \wedge \neg Q(2, x) \implies 5 < x - 1$ evaluates to True for at least one entity $x \in U$:

- $x = 1: P(1) \wedge \neg Q(2, 1) \implies 5 < 0 \rightarrow \text{False}$
- $x = 2: P(2) \wedge \neg Q(2, 2) \implies 5 < 1 \rightarrow \text{False}$
- $x = 3: P(3) \wedge \neg Q(2, 3) \implies 5 < 2 \rightarrow \text{False}$
- $x = 4: P(4) \wedge \neg Q(2, 4) \implies 5 < 3 \rightarrow \text{True}$
- $x = 5: P(5) \wedge \neg Q(2, 5) \implies 5 < 4 \rightarrow \text{True}$

- since the quantified formula holds for 4, 5 $\in U$, F evaluates to True, i.e. we have $\mathcal{A} \models F$



Notes:

Notes:

Group Exercise 08-02

2/2

- Evaluate the truth value of the first-order predicate logic formula F

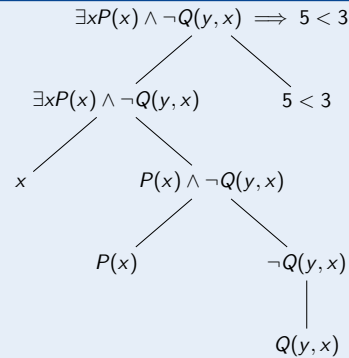
$$\exists x P(x) \wedge \neg Q(y, x) \implies 5 < f(x)$$

for the structure $\mathcal{A}$ given on the previous slide. Additionally consider assignment $x \leftarrow 4$.

- we **substitute free variable** y with $2 \in U$. We **substitute free variable** x in second part of the formula with $4 \in U$.
- we apply **semantics of function** $f(x) := x - 1$
- we test whether $P(x) \wedge \neg Q(2, x)$ evaluates to True for at least one $x \in U$:

- $x = 1: P(1) \wedge \neg Q(2, 1) \rightarrow \text{True}$
- $x = 2: P(2) \wedge \neg Q(2, 2) \rightarrow \text{True}$
- $x = 3: P(3) \wedge \neg Q(2, 3) \rightarrow \text{True}$
- $x = 4: P(4) \wedge \neg Q(2, 4) \rightarrow \text{False}$
- $x = 5: P(5) \wedge \neg Q(2, 5) \rightarrow \text{False}$

- since $5 < 3$ is False, we have $\text{True} \implies \text{False}$, i.e. F evaluates to False, i.e. we have $\mathcal{A} \not\models F$



Equivalences for quantifiers

- we say that two formulas F_1 and F_2 in first-order predicate logic are **logically equivalent** ($F_1 \equiv F_2$) iff $F_1 \iff F_2$ is a tautology, i.e. if **for any structure** $\mathcal{A}$ we have

$$\mathcal{A}(F_1) = \mathcal{A}(F_2)$$

- for formulas α and β (where x is not a free variable in β), the following **logical equivalences** hold

equivalences for $\wedge$ and $\vee$

- $(\forall x \alpha) \wedge \beta \equiv \forall x (\alpha \wedge \beta)$
- $(\forall x \alpha) \vee \beta \equiv \forall x (\alpha \vee \beta)$
- $(\exists x \alpha) \wedge \beta \equiv \exists x (\alpha \wedge \beta)$
- $(\exists x \alpha) \vee \beta \equiv \exists x (\alpha \vee \beta)$

equivalences for $\implies$

- $\exists x \alpha \implies \beta \equiv \forall x (\alpha \implies \beta)$
- $\forall x \alpha \implies \beta \equiv \exists x (\alpha \implies \beta)$
- $\beta \implies \forall x \alpha \equiv \forall x (\beta \implies \alpha)$
- $\beta \implies \exists x \alpha \equiv \exists x (\beta \implies \alpha)$

we can use those equivalences to **enlarge the scope of quantifiers**

Notes:

Notes:

De Morgan's law for quantifiers

- **universal quantifier** can be viewed as **conjunction over universe** of entities e_i

$$\forall x P(x) \equiv P(e_1) \wedge P(e_2) \wedge \dots$$

- **existential quantifier** corresponds to **disjunction over universe** of entities e_i , i.e.

$$\exists x P(x) \equiv P(e_1) \vee P(e_2) \vee \dots$$

- we can apply **De Morgan's law** to quantifiers, i.e.

$$\neg \forall x P(x) \equiv \exists x \neg P(x)$$

$$\neg \exists x P(x) \equiv \forall x \neg P(x)$$

- a further application of De Morgan's law yields the following **equivalences between quantified formulas**

$$\neg \exists x \neg P(x) \equiv \forall x P(x)$$

$$\neg \forall x \neg P(x) \equiv \exists x P(x)$$

- using De Morgan's law, we can **move negations into the quantifier scope**

Satisfiability in first-order logic

- **formula F in propositional logic** is satisfiable iff there is an assignment of variables such that F evaluates to True

- sentence F in first-order predicate logic is **satisfiable** iff $\exists \mathcal{A}$ such that $\mathcal{A}(F) = \text{True}$

- formula F with **free variables** x_i is satisfiable iff sentence $\exists x_1 \exists x_2 \dots \exists x_k \alpha$ is satisfiable

- two formulas F_1 and F_2 are **equisatisfiable** iff F_1 is satisfiable whenever F_2 is satisfiable

- sentence F in first-order predicate logic is **logically valid** iff it is satisfiable **for every structure**

- formula F with free-variables x_i is **logically valid** iff sentence $\forall x_1 \forall x_2 \dots \forall x_k \alpha$ is logically valid

examples

1. $\exists x P(x) \wedge \neg Q(y, x) \implies 5 < f(x)$ is **satisfiable**
2. $\exists x P(x) \wedge \neg P(x)$ is **not satisfiable**
3. $\forall y P(x, y)$ and $\exists x \forall y P(x, y)$ are **equisatisfiable**
4. $\forall x P(x) \vee \neg P(x)$ is **logically valid**
5. $\forall x P(x) \vee \forall x \neg P(x)$ is **not logically valid (but satisfiable)**

equisatisfiability $\neq$ logical equivalence

α not satisfiable $\implies \alpha$ not logically valid

α logically valid $\iff \neg \alpha$ not satisfiable

Notes:

Notes:

Decidability of first-order satisfiability

- ▶ propositional satisfiability is **decidable** (e.g. by truth-table enumeration) but **NP-complete** → L06
- ▶ **FOPL satisfiability is not decidable**, i.e. there is no algorithm that – for any formula – terminates and yields correct answer → cf. L03, Halting problem for Turing machines → D Hilbert and W Ackermann, 1928 → A Church 1936 → AM Turing, 1937
- ▶ but (1): for **some formulas** we can algorithmically determine whether they are satisfiable
- ▶ e.g. propositional logic is **decidable subset** of FOPL
- ▶ but (2): **FOPL unsatisfiability is semi-decidable**, i.e. we can write an algorithm that terminates and yields the correct answer for any unsatisfiable formula
- ▶ we can transform formula to **equisatisfiable, quantifier-free formula in CNF**



David Hilbert, 1862 – 1943

image credit: Wikimedia Commons, public domain

Notes:

Prenex Normal Form

- ▶ formula in predicate logic is in **prenex normal form (PNF)** iff it consists of a prefix of quantifiers with bound variables x_i and a quantifier-free suffix, e.g.

$$\forall x_1 \exists x_2 \dots \forall x_k \phi$$

where ϕ is a **quantifier-free formula** (called “matrix”)

- ▶ using quantifier equivalences and De Morgan’s law, we can transform any formula F into **logically equivalent formula F' in PNF**
- ▶ prenex normal form is important **canonical form** for formulas in first-order predicate logic
- ▶ basis for further normal forms that allow to **check satisfiability** and perform **logical inference**

example

the following formulas are in **prenex normal form**

1. $\forall x_1 \exists x_2 \forall x_3 \forall x_4 P(x_1) \wedge Q(x_2, x_3) \wedge P(f(x_4))$
2. $\exists x_1 \forall x_2 (P(x_1) \implies Q(x_2))$

the following formulas are **not in prenex normal form**

1. $\forall x_1 \exists x_2 \implies \exists x_3 P(x_1, x_2) \wedge Q(x_3)$
2. $\neg \forall x_1 \exists x_2 \forall x_3 \forall x_4 P(x_1) \wedge Q(x_2, x_3) \wedge P(f(x_4))$

Notes:

Skolem Normal Form

- consider formula in PNF with existential quantifier

$$\forall x_1 \forall x_2 \forall x_3 \exists y P(x_1, x_2, x_3, y)$$

- we can **eliminate existential quantifier** by adding a **skolem function** that replaces bound variable y by function f of variables bound by universal quantifier, i.e.

$$\forall x_1 \forall x_2 \forall x_3 P(x_1, x_2, x_3, f(x_1, x_2, x_3))$$

- if there are no variables bound to universal quantifier, we introduce a **skolem constant**, e.g. we replace $\exists y P(y)$ by $P(c)$

- idea: replace “for every x there exists y such that $P(x, y)$ ” by “for every x we have $P(x, f(x))$ for a suitable function f with $f(x) = y$ ”

- skolemized formula is equisatisfiable** (but not logically equivalent)
- basis to **transform formulas** for which we want to check satisfiability



Albert Thoralf Skolem
1887 – 1963

image credit: Wikimedia Commons, public domain

Conjunctive Normal Form

- we can use skolemization and logical equivalences to transform formula in first-order predicate logic into **equisatisfiable formula in Conjunctive Normal Form**

- consider first-order logic formula in prenex normal formal

$$\forall x \neg \forall y (P(x) \implies Q(c, y))$$

- use logical equivalence $X_0 \implies X_1 \equiv \neg X_0 \vee X_1$ to **eliminate implications** (and, if necessary, biconditionals)

$$\forall x \neg \forall y \neg P(x) \vee Q(c, y)$$

- use De Morgan's law to **move negations into quantifiers**

$$\forall x \exists y P(x) \wedge \neg Q(c, y)$$

- introduce skolem function f to **eliminate existential quantifiers** and obtain

$$\forall x P(x) \wedge \neg Q(c, f(x))$$

- resulting formula is **equisatisfiable but not logically equivalent** to original formula and the **matrix is in CNF**

Notes:

Notes:

Group Exercise 08-03

- Transform the following formula into an equisatisfiable formula in CNF.

$$\forall x [\forall y (Animal(y) \implies Loves(x, y)) \implies \exists y (Loves(y, x))]$$

1. we first **eliminate implications**

$$\forall x [\neg(\forall y (\neg Animal(y) \vee Loves(x, y)) \vee \exists y (Loves(y, x)))]$$

2. we **move in negations** and **rename variables**

$$\forall x [\exists y (Animal(y) \wedge \neg Loves(x, y)) \vee \exists z (Loves(z, x))]$$

3. we use skolem functions f, g to **remove existential quantifiers**

$$\forall x [Animal(f(x)) \wedge \neg Loves(x, f(x))] \vee Loves(g(x), x)$$

4. we use the **distributivity over $\wedge$** and obtain formula where matrix is in CNF

$$\forall x [Animal(f(x)) \vee Loves(g(x), x)] \wedge [\neg Loves(x, f(x)) \vee Loves(g(x), x)]$$

Herbrand's theorem

- consider first-order predicate logic formula F in CNF
- **trick**: transform into (possibly infinite) **equisatisfiable conjunction of variable-free propositions**
- **Herbrand universe** $T(F)$ is set of all variable-free terms that can be constructed from **function symbols and constants** in F
- if F does not contain constant, add arbitrary constant c
- **Herbrand expansion** $E(T)$ is **set of propositional formulas** that can be constructed by substituting all variables in F by terms from $T(F)$

Herbrand's theorem

Formula F in Skolem Normal Form is unsatisfiable iff there is a finite subset of terms in the Herbrand expansion $E(T)$ that is propositionally unsatisfiable.

example 1

- consider formula in CNF
$$\forall x \forall y P(x) \wedge \neg Q(c, y)$$
- we have the **set of clauses**
$$\{ P(x), \neg Q(c, y) \}$$
- **Herbrand universe** is $\{c\}$
- **Herbrand expansion** is $\{P(c), \neg Q(c, c)\}$

example 2

- consider formula in CNF
$$\forall x \forall y P(x) \wedge \neg Q(y, f(y))$$
- we have the **set of clauses**
$$\{ P(x), \neg Q(y, f(y)) \}$$
- Herbrand universe is $\{c, f(c), f(f(c)), \dots\}$
- **Herbrand expansion is infinite** and contains
$$\{P(c), \neg Q(c, c), P(f(c)), \neg Q(c, f(c)), P(f(f(c))), \neg Q(c, f(f(c))), \dots\}$$

Notes:

Notes:

DPLL for first-order predicate logic?

- ▶ to prove unsatisfiability of F we can **search for unsatisfiable subsets** of Herbrand expansion $E(F)$
- ▶ we can incrementally add terms from Herbrand expansion and test for satisfiability (e.g. using truth table enumeration or DPLL) → *Gilmore's algorithm*
- ▶ algorithm must (eventually) terminate iff formula F is **not satisfiable** (i.e. iff $\neg F$ is logically valid)
- ▶ for satisfiable formula with infinite Herbrand expansion **algorithm may never terminate**
- ▶ provides a **semi-decision procedure** for unsatisfiability problem in first-order predicate logic

algorithm

```
procedure DPLL-FOPL( $F$  (in CNF))  
   $E(F) \leftarrow \{C_1, C_2, \dots\}$  (Herbrand expansion)  
   $F = \{C_1, C_2\}$   
   $k \leftarrow 2$   
  for  $C_i$  in  $C_3, \dots$ , do  
    if DPLL( $F$ ) = False then  
      return False  
    else  
      add_clause( $F, C_i$ )  
    end if  
  end for  
  return True  
end procedure
```

Logical Inference in first-order logic?

- ▶ consider a knowledgebase KB in **first-order logic**, e.g.

$$\begin{aligned}\forall x(P(x) \implies Q(x, y)) \\ \forall x(\forall y(P(x) \wedge Q(x, y) \implies R(x, y))) \\ P(z)\end{aligned}$$

- ▶ how can we **prove given theorem α** , e.g.

$$R(z, y)$$

- ▶ **truth table enumeration** does not work, as truth table can be infinite!
- ▶ in general, **logical inference in FOPL is undecidable**, i.e. there is no algorithm, which can decide $KB \implies \alpha$ for any KB and α **in finite number of steps**
- ▶ can be reduced to **Halting problem** → *L04 - Turing Machines and Computability*



Kurt Gödel, 1906 – 1971

Gödel's completeness theorem

Every logically valid formula in first-order logic is provable. → *K Gödel, 1929*

image credit: Wikimedia Commons, public domain

Notes:

Notes:

Rules of inference for quantifiers

- ▶ can we apply **rules of inference** to formulas with quantifiers $\forall$ and $\exists$?
- ▶ **instantiation rules:** we can eliminate quantifiers by adding formulas for specific entities

universal and existential instantiation

$$\frac{\forall x \text{Likes}(x, \text{"Icecream"})}{\text{Likes}(\text{"Ingo"}, \text{"IceCream"})} \quad \frac{\exists x \text{Likes}(x, \text{"Icecream"})}{\text{Likes}(\text{"Ingo"}, \text{"IceCream"})}$$

- ▶ **generalization rules:** we can eliminate formulas for specific entities by introducing quantifiers

universal and existential generalization

$$\frac{\begin{array}{l} \text{Likes}(\text{"Ingo"}, \text{"Ice cream"}) \\ \text{Likes}(\text{"Goran"}, \text{"Ice cream"}) \\ \dots \end{array}}{\forall x \text{Likes}(x, \text{"Ice cream"})} \quad \frac{\text{Likes}(\text{"Ingo"}, \text{"Ice cream"})}{\exists x \text{Likes}(x, \text{"Ice cream"})}$$

universal instantiation

$$\frac{\forall x P(x)}{P(e)}$$

where we substitute x by any entity e from universe

existential instantiation

$$\frac{\exists x P(x)}{P(e)}$$

where we substitute x by entity e for which P holds

universal generalization

$$\frac{P(e_i)}{\forall x P(x)}$$

where P holds for any entity e_i in the universe

existential generalization

$$\frac{P(e)}{\exists x P(x)}$$

where e is one entity in the universe

Resolution for first-order logic?

- ▶ for propositional logic, **resolution rule** is complete, i.e. we can use it to **prove any theorem**
- ▶ can we **generalize resolution to first-order logic**?
- ▶ consider following knowledgebase (in PNF)

$$\forall x [\text{Man}(x) \implies \text{Mortal}(x)]$$

$$\text{Man}(\text{Sokrates})$$

- ▶ can we apply resolution to infer $\text{Mortal}(\text{Sokrates})$?
- ▶ yes, iff we ...
 1. apply **universal instantiation** to eliminate $\forall$
 2. **substitute** x by entity Sokrates
- ▶ assuming that free variables are **implicitly universally quantified**, we can write $P(x)$ instead of $\forall x P(x)$

resolution in propositional logic

$$\frac{\begin{array}{l} X_0 \vee \dots \vee X_k \vee C \\ Y_0 \vee \dots \vee Y_l \vee \neg C \end{array}}{X_0 \vee \dots \vee X_k \vee Y_0 \vee \dots \vee Y_l}$$

resolution in first-order logic?

$$\frac{\forall x \neg \text{Man}(x) \vee \text{Mortal}(x)}{\text{Man}(\text{Sokrates})}$$

$$?$$

resolution in first-order logic?

$$\frac{\neg \text{Man}(x) \vee \text{Mortal}(x)}{\text{Man}(\text{Sokrates})}$$

$$?$$

Notes:

Notes:

Notes:

Practice Session

- ▶ we demonstrate the **Z3 Theorem Prover** in python
- ▶ we use Z3 to find **satisfying assignments** for propositional formulas
- ▶ we use Z3 to **prove theorems in propositional and first-order logic**



image credit: Microsoft, fair use

```
Microsoft Z3 Theorem Prover
May 18, 2023

1. import z3
2. from z3 import Int, Bool, Decl, Solver, and
3. s = Solver()

Propositional Logic
Let us test the resolution algorithm using the example from group exercise 03-05:

1. s1, s2, s3 = z3.BoolSort('s1', 's2', 's3')
2. s1 = z3.BoolSort('s1', 's2', 's3')
3. s2 = z3.BoolSort('s1', 's2', 's3')
4. s3 = z3.BoolSort('s1', 's2', 's3')

To prove a theorem alpha based on a knowledgebase kb, we can use the following function:

1. def prove_alpha(kb, alpha):
2.     s = Solver()
3.     s.add(kb)
4.     s.add(alpha)
5.     return s.check()

1. s1, s2, s3 = z3.BoolSort('s1', 's2', 's3')
2. s1 = z3.BoolSort('s1', 's2', 's3')
3. s2 = z3.BoolSort('s1', 's2', 's3')
4. s3 = z3.BoolSort('s1', 's2', 's3')
5. prove_alpha(s1, s2, s3)
```

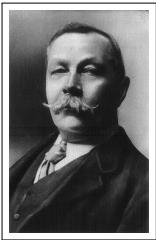
practice session

see notebook 08-01 in gitlab repository at

→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

In summary ...

- ▶ we studied **logical inference** in propositional and first-order logic, i.e. the problem of **proving theorems** based on a knowledgebase
- ▶ for propositional logic, satisfiability and inference is **decidable** and can be solved with **resolution-based algorithms**
- ▶ for first-order logic, unsatisfiability and inference are **semi-decidable**
- ▶ using Herbrand's theorem, **we can use DPLL to develop semi-decision procedure** for first-order logic
- ▶ **resolution-based algorithms** can be generalized to first-order logic, but may not terminate if $KB \not\models \alpha$
- ▶ important foundation for **symbolic AI** and **expert systems**



Sir Arthur Conan Doyle
1859 – 1930

image credit: Wikimedia Commons, public domain

logical inference

“He was, I take it, the most perfect reasoning and observing machine that the world has seen [...]” → *Sherlock Holmes, A Scandal in Bohemia*

Notes:

Notes:

Exercise sheet 04

- ▶ **fourth exercise sheet** will be released today
- ▶ solutions are due **June 2nd** (via WueCampus)
- ▶ present your solution to earn bonus points



Machine Learning for Networks
SoSe 2022

Prof. Dr. Ingo Scholtes
Chair of Informatics XV
University of Würzburg

Exercise Sheet 01

Published: May 4, 2022
Due: May 11, 2022
Total points: 70

Please upload your solutions to WueCampus as a scanned document (image format or pdf), a typeset PDF document, or as a Jupyter notebook.

1. Computing connected components in graphs

(a) Implement Tarjan's algorithm to compute the (strongly) connected components in a (directed) network. Apply your algorithm to a directed example network that has multiple strongly connected components. [2P]

(b) For a Laplacian matrix L of an undirected graph G with n nodes consider the sequence $\lambda_1 = 0 \leq \lambda_2 \leq \dots \leq \lambda_n$ of eigenvalues in ascending order. Generate a sequence of undirected networks with $n = 20$ nodes and different numbers of connected components from one to 20. Calculate the eigenvalue sequences of the corresponding Laplacians. What do you observe? Can you explain your observation? [2P]

(c) Repeat the experiment above using the sorted sequence of eigenvalues of the adjacency matrix. [2P]

(d) Use your finding from the previous tasks to implement a python function that uses the Laplacian matrix to calculate the number of connected components in an undirected graph. Test your function in an example network. What is the computational complexity of your function? [2P]

2. Density-based Clustering with DBSCAN

Cluster detection, i.e. identifying groups of objects more similar to each other than to objects in other groups, is an important unsupervised machine learning task for collections of data points in a Euclidean space. Considering that similarities between points in Euclidean space can be represented by sets, graph-based algorithms have been successfully applied to this problem. A well-known example is DBSCAN, a density-based clustering algorithm that uses connected components in a graph to identify clusters based on contiguous regions with high point density. Consider the following paper (available on WueCampus) and answer the questions below.

M. Ester, H.P. Kriegel, J. Sander, X. Xu: A density-based algorithm for discovering clusters in large spatial databases with noise. In: KDD'96: Proceedings of the Second International Conference on Knowledge Discovery and Data Mining, pp. 226-231, August 1996.

(a) Give a pseudocode implementation of DBSCAN and explain the following aspects of the algorithm: [2P]

- Explain how the parameter ϵ is used to represent the data points in terms of a graph.
- Explain how the parameter s influences the categorization of nodes as core, border, and noise nodes.
- Explain how we can apply Tarjan's algorithm to detect clusters.
- Discuss how the choice of the parameter s influences the number of detected clusters.

(b) Implement the algorithm in python and test it using synthetic data generated by the function `sklearn.datasets.make_moons` in sklearn. [2P]

(c) Investigate for which values of ϵ the algorithm returns a reasonable cluster structure and compare the performance to k -means clustering (e.g. using the implementation included in sklearn). [2P]

Self-study questions

1. Is $\forall x \exists y \exists z \forall f P(f(x), y)$ a valid formula in first-order logic?
2. Consider a variation of first-order predicate logic where all predicates and functions must be of order $k = 0$. How expressive is this variation and how does it compare to propositional logic?
3. Consider $F = \forall x \exists y P(x, y, f(z))$. Give an example for $\mathcal{A} \models F$ and $\mathcal{A}' \not\models F$.
4. Show that quantifiers of the same type are commutative, i.e. $\forall x \forall y R(x, y) \equiv \forall y \forall x R(x, y)$ and $\exists x \exists y R(x, y) \equiv \exists y \exists x R(x, y)$.
5. Show that quantifiers of different type are not commutative, i.e. $\forall x \exists y R(x, y) \not\equiv \exists x \forall y R(x, y)$.
6. Give a model for the formula $\exists y \forall x P(x) \wedge Q(y)$.
7. Use De Morgan's law and equivalences for quantifiers to bring the formula $F = \neg \forall x P(x) \wedge \neg \exists y Q(x, y)$ into a logically equivalent formula in prenex normal form.
8. Give an example for a satisfiable formula in first-order predicate logic that is not logically valid.
9. Are the the two formulas $\exists x P(x)$ and $\nexists x \neg P(x)$ equisatisfiable?
10. Consider the tautology $\forall x \exists y P(x, y) \iff \exists f \forall x P(x, f(x))$ that is the basis for skolemization. Explain why the skolemized form (without $\exists$) is equisatisfiable but not logically equivalent.
11. Give an example for a satisfiable formula in first-order logic where Gilmore's algorithm terminates after a finite number of step.
12. Use the literature references to investigate the details of the unification algorithm mentioned on slide 28.

Notes:

Notes:

References

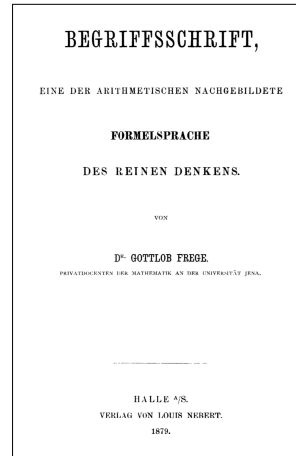
reading list

- ▶ G Frege: **Begriffsschrift. Eine der arithmetischen nachgebildete Formelsprache des reinen Denkens**, Verlag Louis Nebert, 1879
- ▶ C Hartshorne, P Weiss (Eds): **The Collected Papers of C. S. Peirce**, 1987
- ▶ EM Hammer: **Semantics for Existential Graphs**, Journal of Philosophical Logic, Vol. 27, No. 5, 1998
- ▶ D Hilbert and W Ackermann: **Grundzüge der Theoretischen Logik**, Springer, 1928
- ▶ K Gödel: **Über die Vollständigkeit der Axiome des logischen Funktionenkalküls**, Dissertation, 1929
- ▶ A Church: **An unsolvable Problem of Elementary Number Theory**, 1936
- ▶ AM Turing: **On Computable Numbers, with an Application to the Entscheidungsproblem**, 1937
- ▶ JA Robinson: **A Machine-Oriented Logic Based on the Resolution Principle**. Journal of the ACM, 1965
- ▶ U Schöning: **Logik für Informatiker**, Spektrum Akad. Verlag, 2005

acknowledgments

lecture slides partly based on the following books and lectures:

- ▶ M Boehlen: **Introduction to Databases**, University of Zurich
- ▶ U Schöning: **Logik für Informatiker**, Spektrum Akad. Verlag, 2005
- ▶ S Russel and P Norvig: **Artificial Intelligence: A modern Approach**, 2022



Notes: